

Introducing nanochat: The best ChatGPT that \$100 can buy. #1

karpathy announced in Announcements



karpathy last week Maintainer

Ok so we just booted up an 8xH100 box from e.g. [Lambda GPU Cloud](#). This is costing us about ~\$24/hr, so there is no time to lose.

Environment setup

Clone the project:

```
git clone git@github.com:karpathy/nanochat.git
cd nanochat
```



We wish to train the best ChatGPT that \$100 can buy, which we call a "speedrun". Reference the script [speedrun.sh](#), which is designed to just run right away on a blank box start to end. However, in this post I will step through it part by part so that I can comment in detail on all sections of it. We first have to make sure the new&hot [uv](#) project manager is installed. Install uv, create a new virtual environment in `.venv`, get all the dependencies, and activate the environment so that when we type `python` we're using the virtual env python, not the system python:

```
# install uv (if not already installed)
command -v uv &> /dev/null || curl -LsSf https://astral.sh/uv/install.sh | sh
# create a .venv local virtual environment (if it doesn't exist)
[ -d ".venv" ] || uv venv
# install the repo dependencies
uv sync
# activate venv so that `python` uses the project's venv instead of system python
source .venv/bin/activate
```



Next, we need to install Rust/Cargo so that we can compile our custom Rust tokenizer. I know - it's a bit much to have a new/custom tokenizer, unfortunately the Python version in my earlier [minbpe](#) project is way too slow and the huggingface [tokenizers](#) is too bloated and confusing. So we have our own new tokenizer for training (tested to be equal to Python), but we will still use OpenAI's [tiktoken](#) for efficient inference. So here we go, compile our tokenizer:

```
# Install Rust / Cargo
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh -s -- -y
```

```
source "$HOME/.cargo/env"
# Build the rustbpe Tokenizer
uv run maturin develop --release --manifest-path rustbpe/Cargo.toml
```



Train the tokenizer

Next we need the pretraining data so that we can 1) train the tokenizer and 2) pretrain the model. The pretraining data is just the text of a lot of webpages, and for this part we will use the [FineWeb-EDU](#) dataset. Normally, we'd be able to just use `huggingface datasets.load_dataset()`, but I didn't like how it's too heavy, bloated and obscures some very simple logic, so I re-packaged the entire dataset into simple, fully shuffled shards that we can easily and efficiently access at will and re-uploaded the sample-100B version of it as [karpathy/fineweb-edu-100b-shuffle](#). On this page you can also preview example text in the dataset. Each shard is a simple parquet file of about 0.25M characters and takes up about 100MB on disk (gzip compressed). There are 1822 shards in total but we only need 240 of them to train a depth=20 model (more on this later). So let's download all of the data now. This is about ~24GB download here, but it's fairly zippy on a cloud box normally:

```
python -m nanochat.dataset -n 240
```



All of this is by default going into `~/.cache/nanochat`. Once the download is done, let's train our tokenizer, which translates back and forth between strings and sequences of symbols from a codebook. By default we are training a vocab size of $2^{16} = 65,536$ tokens (a nice number), of which a few tokens are reserved as special (to be used for the chat schema later). The training set is 2B characters, which only takes ~1 minute. The training algorithm is identical to the one used by OpenAI (regex splitting, byte-level BPE). See my video on [tokenization](#) for a lot more information. Right after, we can evaluate the tokenizer:

```
python -m scripts.tok_train --max_chars=2000000000
python -m scripts.tok_eval
```



The evaluation tells us that we're achieving a compression ratio of about 4.8 (meaning 4.8 characters of original text become 1 token on average). We can also see a comparison to the GPT-2 and GPT-4 tokenizer. Compared to GPT-2 (which has 50257 tokens), ours is much better across the board in compressing text, except for math by a little bit:

Comparison with GPT-2:							
Text Type	Bytes	GPT-2 Tokens	Ratio	Ours Tokens	Ratio	Relative Diff %	Better
news	1819	404	4.50	375	4.85	+7.2%	Ours
korean	893	745	1.20	712	1.25	+4.4%	Ours
code	1259	576	2.19	492	2.56	+14.6%	Ours
math	1834	936	1.96	966	1.90	-3.2%	GPT-2
science	1112	260	4.28	228	4.88	+12.3%	Ours
fwe-train	4208518	900364	4.67	856883	4.91	+4.8%	Ours
fwe-val	4908443	1059062	4.63	1010352	4.86	+4.6%	Ours

We're not doing so hot compared to GPT-4, but you have to keep in mind that GPT-4 has a much larger vocab size (100,277). In particular, GPT-4 is a lot better in multilingual (FineWeb has a very strong focus on English, so that makes sense!), but also on code and math:

Text Type	Bytes	GPT-4 Tokens	Ratio	Ours Tokens	Ratio	Relative Diff %	Better
news	1819	387	4.70	375	4.85	+3.1%	Ours
korean	893	364	2.45	712	1.25	-95.6%	GPT-4
code	1259	309	4.07	492	2.56	-59.2%	GPT-4
math	1834	832	2.20	966	1.90	-16.1%	GPT-4
science	1112	249	4.47	228	4.88	+8.4%	Ours
fwe-train	4208518	874799	4.81	856883	4.91	+2.0%	Ours
fwe-val	4908443	1029691	4.77	1010352	4.86	+1.9%	Ours

Still, we actually beat GPT-4 by a tiny bit even though we have a lower vocab size on fineweb, because that's the dataset we actually trained on, so our tokenizer matches that document distribution very well (e.g. we might get an edge on compressing English).

Pretraining

Before we kick off pretraining, we need to download one more file that I call the "eval bundle". During pretraining, the script will periodically evaluate the [CORE metric](#). You can see some details in the DCLM paper, but essentially it is a nice, normalized, broad measure of how good the model is across a large number of datasets in autocompletion. Datasets like HellaSwag, jeopardy, bigbench QA wikidata, ARC-Easy/Challenge, copa, commonsense qa, piqa, lambada, winograd, boolq, etc etc (22 total). Download, unzip and place the eval bundle directory into the base directory as

~/cache/nanochat/eval_bundle :

```
curl -L -o eval_bundle.zip https://karpathy-public.s3.us-west-2.amazonaws.com/eval_bundle.zip
unzip -q eval_bundle.zip
rm eval_bundle.zip
mv eval_bundle "$HOME/.cache/nanochat"
```

One more setup I'd advise (though it's optional) is to set up [wandb](#) for seeing nice plots during training. uv already installed wandb for us up above, but you still have to set up an account and log in with:

```
wandb login
```

We can now kick off pretraining! This is the most computationally heavy part, where we are training the LLM to compress internet web text by predicting the next token in the sequence, and where the LLM gains a lot of knowledge about the world:

```
torchrun --standalone --nproc_per_node=8 -m scripts.base_train -- --depth=20
```

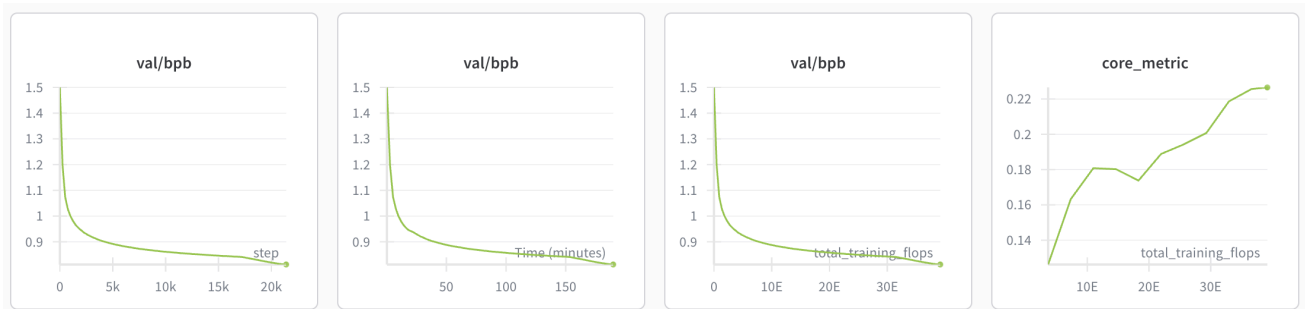
Here we are launching training on 8 GPUs via the [scripts/base_train.py](#) script. We're training a Transformer with 20 layers. By default, each GPU is processing 32 rows of 2048 tokens per forward/backward for a total of $32 \times 2048 = 2 \times 10^5 = 524,288 \approx 0.5\text{M}$ tokens per step of the optimization. If you have wandb set up, append `--run=speedrun` (all training scripts accept it) to set the run name and log to it. When you launch training, you'll see something like this (stripping a bunch of stuff for brevity):



Vocab size: 65,536
num_layers: 20
model_dim: 1280
num_heads: 10
num_kv_heads: 10
Tokens / micro-batch / rank: 32 x 2048 = 65,536
Tokens / micro-batch: 524,288
Total batch size 524,288 => gradient accumulation steps: 1
Number of parameters: 560,988,160
Estimated FLOPs per token: 3.491758e+09
Calculated number of iterations from target data:param ratio: 21,400
Total number of training tokens: 11,219,763,200
Tokens : Params ratio: 20.00
Total training FLOPs estimate: 3.917670e+19
Scaling the LR for the AdamW parameters $\propto 1/\sqrt{(1280/768)} = 0.774597$
Muon: Grouping 80 params of shape torch.Size([1280, 1280]), device cuda:0, dtype torch.float32
Muon: Grouping 20 params of shape torch.Size([1280, 5120]), device cuda:0, dtype torch.float32
Muon: Grouping 20 params of shape torch.Size([5120, 1280]), device cuda:0, dtype torch.float32
Step 00000 | Validation bpb: 3.3013
^[step 00000/21400 (0.00%) | loss: 11.090355 | lrm: 1.00 | dt: 23156.74ms | tok/sec: 22,640 | mfu: 1.00 | total time: 0.00m
step 00001/21400 (0.00%) | loss: 10.808654 | lrm: 1.00 | dt: 649.22ms | tok/sec: 807,569 | mfu: 35.64 | total time: 0.00m
step 00002/21400 (0.01%) | loss: 10.179083 | lrm: 1.00 | dt: 472.29ms | tok/sec: 1,110,094 | mfu: 48.99 | total time: 0.00m
step 00003/21400 (0.01%) | loss: 9.449214 | lrm: 1.00 | dt: 487.47ms | tok/sec: 1,075,523 | mfu: 47.47 | total time: 0.00m
step 00004/21400 (0.02%) | loss: 8.903216 | lrm: 1.00 | dt: 487.57ms | tok/sec: 1,075,308 | mfu: 47.46 | total time: 0.00m
step 00005/21400 (0.02%) | loss: 8.531662 | lrm: 1.00 | dt: 482.58ms | tok/sec: 1,086,417 | mfu: 47.95 | total time: 0.00m
step 00006/21400 (0.03%) | loss: 8.231589 | lrm: 1.00 | dt: 487.21ms | tok/sec: 1,076,113 | mfu: 47.49 | total time: 0.00m
step 00007/21400 (0.03%) | loss: 7.993080 | lrm: 1.00 | dt: 484.10ms | tok/sec: 1,083,014 | mfu: 47.80 | total time: 0.00m
step 00008/21400 (0.04%) | loss: 7.803373 | lrm: 1.00 | dt: 488.17ms | tok/sec: 1,073,989 | mfu: 47.40 | total time: 0.00m
step 00009/21400 (0.04%) | loss: 7.627318 | lrm: 1.00 | dt: 484.78ms | tok/sec: 1,081,486 | mfu: 47.73 | total time: 0.00m
step 00010/21400 (0.05%) | loss: 7.491893 | lrm: 1.00 | dt: 487.03ms | tok/sec: 1,076,511 | mfu: 47.51 | total time: 0.00m
step 00011/21400 (0.05%) | loss: 7.354157 | lrm: 1.00 | dt: 487.10ms | tok/sec: 1,076,334 | mfu: 47.50 | total time: 0.01m
step 00012/21400 (0.06%) | loss: 7.246406 | lrm: 1.00 | dt: 487.99ms | tok/sec: 1,074,383 | mfu: 47.42 | total time: 0.02m
step 00013/21400 (0.06%) | loss: 7.159368 | lrm: 1.00 | dt: 486.56ms | tok/sec: 1,077,540 | mfu: 47.55 | total time: 0.02m

We see that the Transformer has 1280 channels and 10 heads in Attention, each of $\text{dim}=128$. It has $\sim 560\text{M}$ parameters. In order to meet [Chinchilla scaling law](#) recommendations, this means we want $560\text{M} \times 20 \sim 11.2\text{B}$ tokens to train on. As each step of the optimization is 524,288 tokens, this means $11.2\text{B} / 0.5\text{M} \sim 21400$ iterations. Taking the estimated number of FLOPs per token and multiplying by total number of tokens tells us that this will be a $\sim 4\text{e}19$ FLOPs capability model. The learning rate is automatically scaled down as $1/\sqrt{\text{dim}}$, as larger models prefer smaller learning rates. We're using Muon to optimize the matrices and AdamW to optimize the embedding and unembedding. There are no other trainable parameters (biases, rmsnorms params, etc.) in this model. Training will periodically report the "Validation bpb", which is bits per byte on the validation dataset. Bits per byte is a much better measure than just the typical cross-entropy loss, because it further normalizes the loss on each token by the number of bytes of that token, making the metric tokenizer-invariant. So if you have a tokenizer with a small vocab size or a big one, this number will be comparable, unlike raw cross-entropy loss. Notice that each step is taking about 0.5s, lr_m is the learning rate decay multiplier (it will linearly ramp down to 0 near the end of training), the reported MFU (model flops utilization) looks good at almost $\sim$ half, meaning that we are utilizing a lot of the bfloat16 compute available to us.

We now wait for about 3 hours for $4\text{e}19$ FLOPs to elapse... You should see something like this in your wandb plots:



bpb going down over time is good (the model is predicting the next token more accurately). In addition, the CORE score is going up. Instead of just approximated metrics, we can evaluate the model more fully as:

```
torchrun --standalone --nproc_per_node=8 -m scripts.base_loss
torchrun --standalone --nproc_per_node=8 -m scripts.base_eval
```



We see that we reach train/val bits per byte (bpb) of ~ 0.81 and the CORE metric goes up to 0.22. For comparison, the eval bundle contains the GPT-2 model CORE scores. In particular, CORE of 0.22 is a little bit more than GPT-2 large (at 0.21) but a little bit less than GPT-2 xl (i.e. "the" GPT-2, at 0.26). The model at this point is a fancy autocomplete, so we can run a few prompts to get a sense of the knowledge stored in the model. The file `base_loss.py` runs these. The prompts are:

```
prompts = [
    "The capital of France is",
    "The chemical symbol of gold is",
    "If yesterday was Friday, then tomorrow will be",
    "The opposite of hot is",
    "The planets of the solar system are:",
    "My favorite color is",
    "If  $5x + 3 = 13$ , then  $x$  is",
]
```



And the completed text is:

The capital of France is Paris. It is the largest city in France and the second largest city in Europe

The chemical symbol of gold is Au. The chemical symbol of silver is Ag. The chemical symbol of copper is

If yesterday was Friday, then tomorrow will be Saturday. If yesterday was Monday, then tomorrow will be Monday. If yesterday was

The opposite of hot is cold. The opposite of hot is cold. The opposite of hot is cold.

The planets of the solar system are: Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, Neptune,

My favorite color is red. It is the color of the sun, the color of the sky,

If $5x + 3 = 13$, then x is a positive integer.



So the model knows about Paris (France), that Au is gold, that Saturday follows Friday, that "cold" is the opposite of "hot", and even the planes of the solar system. However, it's not so sure about the color of the sky yet, or how to do simple maths. Still, not too bad for a model trained for \$72. The inference uses a custom `Engine` class, which uses KV Caching for efficient inference, as well as a simple implementation of the two common inference stages: prefill and decode. Our `Engine` class also supports tool use (of Python interpreter), which will be useful when training on GSM8K (more on that later).

Midtraining

Next up is midtraining, which further finetunes the model on [smol-SmolTalk](#). Everything is algorithmically identical to pretraining, but the dataset now becomes conversations, and the model adapts itself to the new special tokens that now structure multi-turn Conversation objects. Each conversation now looks something like this, loosely following the OpenAI [Harmony chat format](#):

```
<|bos|>
<|user_start|>What is the color of the sky?<|user_end|>
<|assistant_start|>Red. Wait, possibly blue. I'm not sure.<|assistant_end|>
<|user_start|>lol<|user_end|>
<|assistant_start|>...etcetc
```



The tokens rendered as `<|example|>` are special tokens, following the format of OpenAI special tokens. The midtraining stage is quite useful for a number of adaptations in the model:

- the model learns the special tokens associated with multi-turn conversation (Except for the `<|bos|>` token that delimits documents, these were absent during pretraining of the base model)
- the model adapts itself to the data distribution of conversations instead of internet documents
- very important to us, we have to teach the model to take Multiple Choice quizzes, because at this small of a model scale, the model doesn't learn it just from random internet data. In particular, the model has to learn the algorithm of associating a few choices to a few letters (e.g. ABCD) and then emitting the choice that is correct. We do this by just mixing in 100K multiple choice questions from the MMLU auxiliary_train split. To be clear, the issue is not that the model doesn't have the knowledge, it's that it doesn't understand how Multiple Choice works to surface that knowledge. This is important for us because a lot of common model evaluations (e.g. MMLU) take the form of multiple choices quizzes.

- you can teach the model to use various tools. For us, we need to teach the model to use the Python interpreter by putting Python commands in between the special tokens <|python_start|> <|python_end|>. This will be useful for completing GSM8K problems later.
- you can target many other adaptations during midtraining, e.g. context length expansion (not explored yet)

Our midtraining mixture looks like this by default:

```
train_dataset = TaskMixture([
    SmolTalk(split="train"), # 460K rows of general conversations
    MMLU(subset="auxiliary_train", split="train"), # 100K rows of multiple choice problem
    GSM8K(subset="main", split="train"), # 8K rows teaching simple math and (calculator)
]) # total: 460K + 100K + 8K = 568K rows
```



And we kick it off as follows:

```
torchrun --standalone --nproc_per_node=8 -m scripts.mid_train
```



This run only takes about 8 minutes, a lot shorter than pretraining at ~3 hours. Now that the model is a proper Chat model and it can take on the role of an Assistant answering User queries, we can evaluate it:

```
torchrun --standalone --nproc_per_node=8 -m scripts.chat_eval -- -i mid
```



We get the following results for the model at this stage:

```
- ARC-Easy: 0.3561
- ARC-Challenge: 0.2875
- MMLU: 0.3111
- GSM8K: 0.0250
- HumanEval: 0.0671
- ChatCORE metric: 0.0730
```

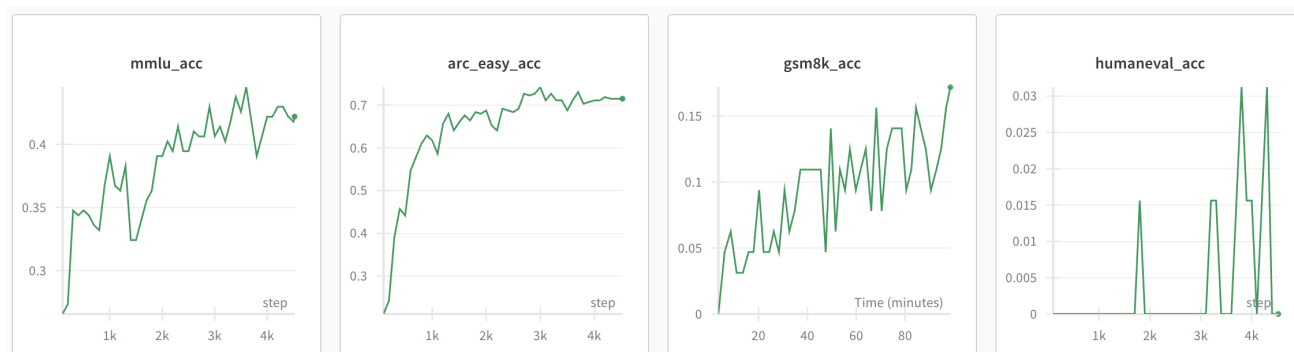


We see that:

- **world knowledge.** The first 3 ([ARC-E/C](#) and [MMLU](#)) are all multiple choice quizzes, measuring the model's world knowledge across various fields. Random chance is about 25% as there are 4 choices (A,B,C,D), so our model is already doing better than that. (Multiple choice is pretty hard for models this small.)
- **math.** [GSM8K](#) are grade school math problems. Baseline performance here is 0% because the model has to write down the actual answer number. Our performance as of right now is still not that strong, solving only 2% of problems.
- **code** [HumanEval](#) is a Python coding benchmark, again with a random baseline of 0%.
- **ChatCORE** is my attempt to replicate what the CORE score is doing for base models and extend it to Chat. That is, we take all the metrics above, subtract baseline performance so that our score is between 0 and 1 (i.e. random model is 0, not 25% on MMLU as an example), and report the mean over all tasks. It's a single number summary of the strength of model right now.

- The evals are still fairly incomplete and there are a lot of other things we could measure but don't yet.

I don't really have a nice graphic to illustrate this step, but here is an example of midtraining a different, bigger model earlier just to give you a sense of what it looks like for these metrics to go up during a finetuning run:



Supervised Finetuning (SFT)

Following midtraining is the Supervised Finetuning (SFT) stage. This is an additional round of finetuning on Conversations, but ideally here you'd cherry pick just the most beautiful/good data, and this is also where you'd do things like safety training (e.g. assistant refusals). Our model isn't even sure about the color of the sky so we're probably safe on the biohazard side of things for now. One domain adaptation that happens here is that SFT stretches out rows of data and pads them, exactly mimicking the test-time format. In other words, examples are not just randomly concatenated into long rows like in pre/mid-training, where it is done for efficiency of training. Fixing this domain mismatch serves as another little "tightening the screws" boost. We can run SFT and re-evaluate:

```
torchrun --standalone --nproc_per_node=8 -m scripts.chat_sft
torchrun --standalone --nproc_per_node=8 -m scripts.chat_eval -- -i sft
```



This again only runs for about 7 minutes and you should notice a small bump in metrics:

```
- ARC-Easy: 0.3876
- ARC-Challenge: 0.2807
- MMLU: 0.3151
- GSM8K: 0.0455
- HumanEval: 0.0854
- ChatCORE metric: 0.0884
```



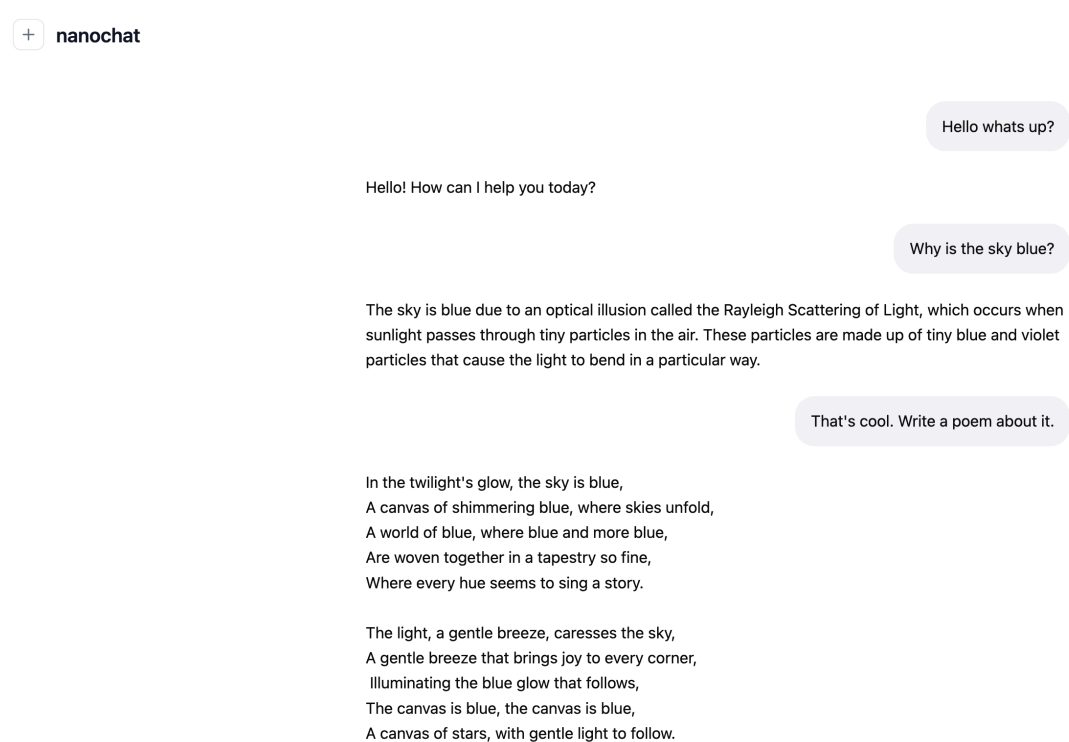
Finally, we can take on the role of a User and talk to our model! We could have already done it after midtraining, but it's a bit nicer here. Talk to it either in your terminal window (line 1), or via the web UI (2):

```
python -m scripts.chat_cli
python -m scripts.chat_web
```



The `chat_web` script will serve the Engine using FastAPI. Make sure to access it correctly, e.g. on Lambda use the public IP of the node you're on, followed by the port, so for example <http://209.20.xxx.xxx:8000/>, etc.

That will gloriously look something like this:



It won't win any physics or poem competitions anytime soon, but again - it seems cool how far we can go with so little budget, and this project is nowhere near tuned enough.

Reinforcement Learning (RL)

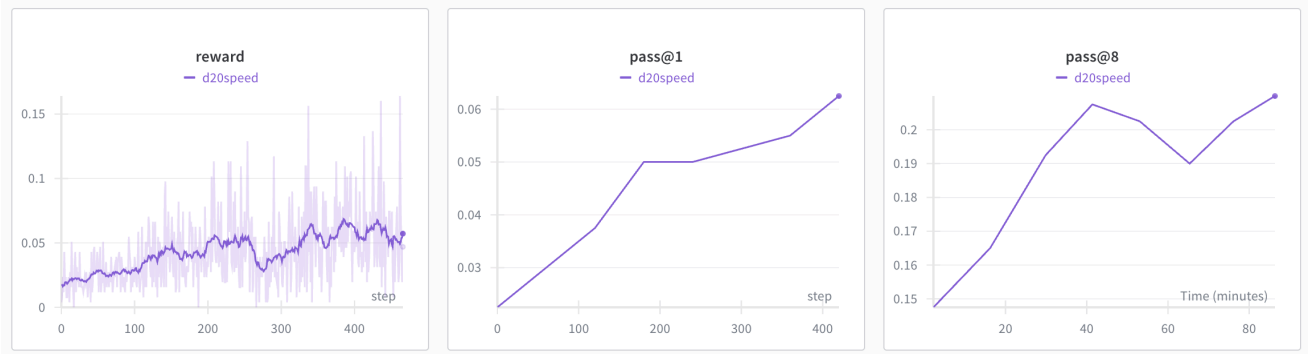
The final stage of the speedrun (though it is commented out by default) is Reinforcement Learning. RLHF is a nice way to gain a few percent of performance and mitigate a lot of model shortcomings that come from the sampling loop itself - e.g. hallucinations, infinite loops, etc. But at our scale these are not a major consideration. That said, of all the datasets we're working with so far, GSM8K is the one that has a clear/objective reward function (the correct answer to a math problem). So we can run the RL (/GRPO) script to hillclimb on the answers directly in a simple RL loop that interleaves sampling and training:

```
torchrun --standalone --nproc_per_node=8 -m scripts.chat_rl
torchrun --standalone --nproc_per_node=8 -m scripts.chat_eval -- -i rl -a GSM8K
```



During RL, the model goes over all GSM8K problems in the training set, samples completions, then we reward them, and train on the ones that got high rewards. We're using a highly simplified GRPO training loop, e.g. we don't use trust regions (throw away reference model and KL regularization), we are on policy (throw away the PPO ratios+clip), we use GAPO style normalization (token-level, not sequence-level normalization), and the advantage is simple reward shift by mean (throw away z-score normalization with dividing by sigma). So we're left with something that looks quite a bit more like REINFORCE, but keeping the GR ("group relative") part in calculating advantages from the rewards. It works ok at this scale and task simplicity. See script for more details.

RL is commented out by default right now because it's not super well-tuned, and we don't have full and general RLHF. We only have RL on GSM8K specifically, which is why we're also restricting the evaluation to only gsm8k with `-a` flag. It also runs for quite a while because reinforcement learning is sucking supervision bits through a straw. E.g. the default settings run for about 1.5 hours and look like this:



We can see that reward goes up (i.e. the model is learning), the accuracy (pass@1) is climbing, and so is pass@8 (i.e. we're given 8 opportunities to get the right answer). It's also promising that $\text{pass@8} \gg \text{pass@1}$, indicating that there is still gap to be claimed here with more RL and more epochs. The improvements are more prominent on larger models, e.g. I ran up to d30 so far. I'm not going to spend as much time on this because honestly this part is not super well-tuned, and it creates a GSM-specific model, not a general chat model.

Report card

The final thing I'd like to point out is the `report.md` file that appears in your project folder. It contains a lot of the details related to the run, as well as a nice summary table at the end:

- Characters: 333,989
- Lines: 8,304
- Files: 44
- Tokens (approx): 83,497
- Dependencies (uv.lock lines): 2,004

Metric	BASE	MID	SFT	RL
CORE	0.2219	-	-	-
ARC-Challenge	-	0.2875	0.2807	-

Metric	BASE	MID	SFT	RL
ARC-Easy	-	0.3561	0.3876	-
GSM8K	-	0.0250	0.0455	0.0758
HumanEval	-	0.0671	0.0854	-
MMLU	-	0.3111	0.3151	-
ChatCORE	-	0.0730	0.0884	-

Total wall clock time: 3h51m

Note that since the support for RL right now is a little bit mixed, I exclude it from the total wall clock time calculation. Up to and including SFT, the whole thing ran in 3h51m, for a total cost of $(3 + 51/60) * 24 = \$92.4$, (with RL this is a bit closer to 5 hours right now). We even have \$8 left for ice cream.

Your turn

With nanochat, you can tune *anything*. Change the tokenizer, change any of the data, tune the hyperparameters, improve the optimization... there are many ideas to try. You may also wish to train bigger models. The codebase is set up to do that quite easily, simply use `--depth` to change the number of layers and everything else is based off of that as the single slider of complexity. For example, the number of channels will grow, the learning rates will adjust, etc. In principle, just by changing the depth you can sweep out an entire miniseries of nanochat. You should also see strictly better results by using a larger depth and waiting longer. The place you'd pass it in is during the `base_train.py` pretraining stage. For example, to get a GPT-2 capability model of CORE about 0.25, d26 is a good number to try. But to train larger models, we now have to tune the max device batch size, e.g. decreasing it from 32 to 16:

```
torchrun --standalone --nproc_per_node=8 -m scripts.base_train -- --depth=26 --
device_batch_size=16
```



The code will notice and automatically compensate, calculating that it needs to now do a gradient accumulation loop of 2 iterations to meet the target desired batch size of 0.5M. To train a d30, we have to decrease it further again:

```
torchrun --standalone --nproc_per_node=8 -m scripts.base_train -- --depth=30 --
device_batch_size=8
```



And so on. Feel free to also read the code, I tried very hard to keep it readable, commented, clean and accessible. And of course, feel free to package it all up and ask your favorite LLM as well, or even simpler, use [DeepWiki](#) from Devin/Cognition to ask questions of this repo. Just change the URL of the repo from github.com to deepwiki.com, i.e. [nanochat DeepWiki](#).

That's it, tune any part of the entire pipeline, re-run, and have fun! Ask any questions here, in the Issues/Discussions, or [on my Discord](#) in the channel #nanochat .

↑ 167

👍 270

😄 22

🎉 70

❤️ 226

🚀 88

👁️ 14

62 comments · 17 replies

Oldest

Newest

Top



aravhawk [last week](#)

What does the performance look like if, say, it was scaled down to FP4 on a RTX Blackwell card?

↑ 1

👍 6

1 reply



ppk9058 [last week](#)

such light weighted model but has high cost to benifit ratio.Great

👁️ 1



alexeichhorn [last week](#)

edited ▼

Thank you for all your work ❤️

It would be cool to turn this into a reasoning model - just to see how much reasoning actually helps in such small models.

↑ 8

👁️ 2

2 replies



ManuelFay [last week](#)

Interesting paper on that if you want:

<https://www.arxiv.org/abs/2509.22193>

Tl;dr is: clearly helps boost performance at isomodel sizes, but the story is the other way around if you are looking at the problem from an inference isoflop perspective (for small models, better to use a larger one that does not use reasoning and produces less tokens)

👍 3



alexeichhorn [last week](#)

Interesting paper on that if you want:

<https://www.arxiv.org/abs/2509.22193>

Tl;dr is: clearly helps boost performance at isomodel sizes, but the story is the other way around if you are looking at the problem from an inference isoflop perspective (for small models, better to use a larger one that does not use reasoning and produces less tokens)

Thanks for sharing this. Very interesting. I guess makes sense when thinking about it.



bquast [last week](#)

I love this work

↑ 4

0 replies



egorsmkv [last week](#)

I am teaching Rustaceans to use Rust in ML right now on Ukrainian Rustcamp and this code will be useful for them. Thanks for open sourcing it!

↑ 1



0 replies



d1hr2uv [last week](#)

can I make one with 10 \$????? I

↑ 3

1 reply



ValtersJZ [5 days ago](#)

edited ▼

For fun? Yes, use a cheap gpu provider/marketplace, I got one 5090 ~\$0.35/hr, batch size 16, depth 16 instead of 20 (random, prob better to keep at 20), add weight checkpointing; 10-30 hours of training should get decent results (got 85% mfu)



Kbediako [last week](#)

wow thank you!

↑ 1

0 replies



sanej [last week](#)

incredible work! [@karpathy](#) you keep pushing the bar higher

↑ 3

0 replies



bakibillah [last week](#)

awesome contribution to the open source community.

↑ 4

0 replies



z4ziggy [last week](#)

Thank you for this. Your code & teaching are very much appreciated ❤️
And please fix the path to speedrun.sh (should be
<https://github.com/karpathy/nanochat/blob/master/speedrun.sh>)

↑ 4

0 replies



rjshxyz [last week](#)

I pray the LLM God Karpathy

(*>人<)

↑ 5

0 replies



erdaltoprak [last week](#)

Thank you openly sharing your knowledge!

↑ 1

0 replies



rimomcosta [last week](#)

Thank you a lot. It would be great if you could share the trained model, as not everyone will be able to spend \$100 on this, so they can at least test the inference.

↑ 7



3

2 replies



samdobson [last week](#)

edited ▼

I'll be sharing the model at <https://huggingface.co/sdobson/nanochat> once training completes, 4 hours from now.

You can watch the run live at <https://api.wandb.ai/links/sjd333-none/dsv4zkij>

EDIT: Run finished and model uploaded. I didn't get quite so good results (unlucky seed?) but it's a lot of fun to play with!

❤️ 16



samdobson [last week](#)

You can chat with the 561M parameter model (after SFT, no RL) at <https://huggingface.co/spaces/sdobson/nanochat>

I modified the inference code to run on CPU, so you can duplicate the space (this one will not support many simultaneous users) and run your own copy of the model for free.

👍 11



eri-bga [last week](#)

Awesome! Now I need to run it on my old PC

↑ 2

0 replies



sampathchanda [last week](#)

Incredible work—been in-love with your from-scratch implementations forever. Great service to open-source research community! 🙌

A couple of clarifications that might help readers:

- After installing `uv`, it'd be helpful to note that users should add it to their `PATH`.
- In the pre-training section, for estimating tokens per optimizer step:

```
step_tokens = num_gpus * batch_size * max_seq_len
```



- The $\sim 4e19$ "capability" figure lines up with Chinchilla's compute-optimal rule:

```
FLOPs ≈ 6 * model_params * training_tokens = 6 * 0.56e9 * 11.2e9 = 3.77e19 ≈ 4e
```



A few questions:

1. What's the roadmap for this repo? I'm interested in contributing and would love pointers on priorities.
2. Do you plan to explore a vision-language variant (VLM), as part of this repo, or a different one (including SFT & RL, since nanoVLM from HF only does pre-training)?
3. Are there plans to incorporate explicit reasoning capabilities (e.g., CoT supervision, tool use, or RL-style reasoning curricula)?

↑ 6

👍 6

0 replies



norhum [last week](#)

Omg I am gonna love reading through this. Thank you so much!

↑ 1

0 replies

32 hidden items

[Load more...](#)



adminvsrm [last week](#)

Great! I will try it in Vietnamese! <https://chuvnsongsong.com/>

↑ 1

0 replies



Karthik-Jothi [last week](#)

Grateful for your incredible contributions to the AI community, Andrej!

↑ 1

0 replies



fslurrehman [last week](#)

Can someone post step by step procedure for setting up:

1. Lambda GPU
2. or. preferably Google Cloud for this?



ffcai [5 days ago](#)

It's pretty straightforward with [Lambda Cloud](#), as outlined in [README](#):

1. Launch an 8×H100 instance at ~\$24/hour.
2. git clone nanochat repo.
3. Run the speedrun script inside a screen session.

The training completed in ~3 hours, costing a little over \$100—I spent some extra time exploring the model and downloading files.



Seqaeon [2 days ago](#)

It's pretty straightforward with [Lambda Cloud](#), as outlined in [README](#):

1. Launch an 8×H100 instance at ~\$24/hour.
2. git clone nanochat repo.
3. Run the speedrun script inside a screen session.

The training completed in ~3 hours, costing a little over \$100—I spent some extra time exploring the model and downloading files.

Please what was your final loss during this training run [@ffcai](#)



ffcai [13 minutes ago](#)

[@Seqaeon](#) very close to Karpathy's numbers in this post.

Base model loss

- train bpb: 0.8147
- val bpb: 0.8120



ShuaixinHuang [5 days ago](#)

感谢有你，感谢分享，很好用



metagenis [5 days ago](#)

Amazing - i am already fine tuning my own model. thank you!

↑ 1

0 replies



ShuaixinHuang [5 days ago](#)

Sure, sure. I'll give it a try. Thank you for your reminder. Your project is a great one. I'm a media person and enthusiast in the field of large models. I've decided to help you promote it. You deserve this promotion.

...

↑ 2

0 replies



endruuu [5 days ago](#)

Love this! Thank you so much!!

Side note:

If yesterday was Friday, then tomorrow will be Saturday.



It should be Sunday.

↑ 2

0 replies



ShuaixinHuang [5 days ago](#)

Sure, I'm currently a freshman in college and a big fan of large models. I really like your open-source project. Could I ask you for some advice on learning about large models?

...

↑ 1

0 replies



sonnxx [5 days ago](#)

Wowwww, thank You.

↑ 1

0 replies



sam-vish [5 days ago](#)

edited ▾

Wow, can I mix both Tinker Finetuning API by Thinking Machines and NanoChat to maximise their use?

↑ 1

0 replies



limin112 [4 days ago](#)

cool

↑ 1

0 replies



FJ0451 [4 days ago](#)

edited ▾

I ran into a problem with two nearly identical machines each with a single 4090, but only one was training correctly. On the working one the samples were sentish-sh after an hour and loss approached 4.0. The other's loss plateaued around 7.8 after many hours, and the samples never got better than repeating "the" and "," characters.

The environments are identical as far as I can tell. I tried a shotgun of random fixes and eventually found that disabling AMP and fused attention fixed things on the "broken" machine.

```
export CUDA_LAUNCH_BLOCKING=1
python - <<'PY'
import os, torch
from torch.backends.cuda import sdp_kernel
sdp_kernel(enable_flash=False, enable_math=True, enable_mem_efficient=False)
torch.backends.cuda.matmul.allow_tf32 = False
torch.backends.cudnn.allow_tf32 = False
print("SDP kernels set to math; TF32 disabled.")
PY
```



I'm not sure if there is something wrong on my system, there is a subtle bug in the training code, or if this is just an unlucky quirk of numerical precision failing in just the right way that it happened to tank training on this one box. But if you find your training loss gets stuck, maybe try turning this off. (Disabling this lowers the speed about 20 to 25 percent.)

UPDATE: If I try different depth models, some do stall out in the same way on the "working" system. All depth models do train correctly setting just `export CUDA_LAUNCH_BLOCKING=1`. The difference between working and not working is obvious even after just one minute, loss ~6.5 vs ~8.0, and the gap gets only wider. So it's something in the early/warmup stages?

↑ 1

0 replies



NE7K [4 days ago](#)

I love this

↑ 1

0 replies



Keith-Hon [3 days ago](#)

Love it!

↑ 1

0 replies



jasonacox [2 days ago](#)

Has anyone attempted this on a [NVIDIA DGX Spark](#)? Will be slower but should work similar to single GPU mode.

↑ 1

1 reply



jasonacox [2 days ago](#)

Discussion already started: [#28](#)

Category



Announcements

Labels

None yet

69 participants

